

Welcome to Interview Kickstart !

Sorting Live Class



Omkar Deshpande

Head of Curriculum @ Interview Kickstart

Contact: Discord, omkar@interviewkickstart.com

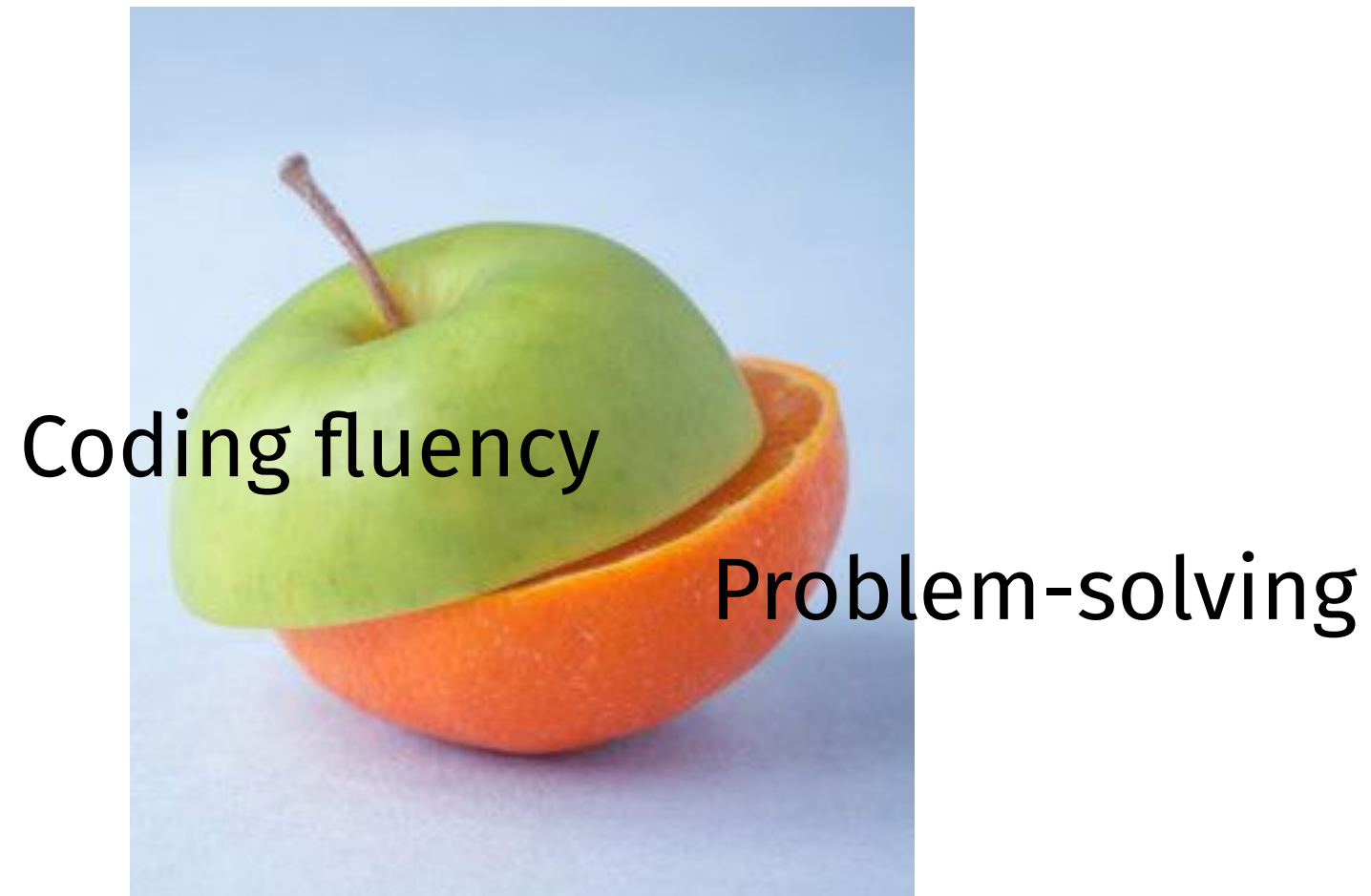
Agenda

- Introduction to Problem-Solving & Coding Patterns
- Interview problems related to Sorting
 - Presorting
 - Extensions of Merge Sort
 - Extensions of Quick Sort
 - Extensions of Heapsort

Lunch break between 12:30 pm and 1 pm PST

(My class usually goes up to 6 hrs since there are lots of questions)

The two halves of interview prep



Problem-solving

A **problem** is a question for which you have no idea (at least initially) what method will work. You *will* feel clueless at the beginning.

Problems are not **exercises**.

Example exercise: $3456789 \times 43289763 = ?$

It is hard, but there is no doubt what method to use.

A problem requires sustained investigation, deploying *problem-solving strategies*.

Mathematical problems, scientific problems, and
computational problems.

The intent of an ideal technical interview

An *algorithm* is the method to solve a computational problem.

Implemented as a *program*.

Can you design a correct and efficient algorithm that solves the problem, and code it up?

Live classes

This tests your problem-solving skills, your coding skills and your knowledge of computer science and engineering fundamentals.

Foundation videos

“But a real interview is a timed test!”



Research suggests timed tests cause math anxiety ★

JO BOALER, PROFESSOR OF MATHEMATICS EDUCATION, STANFORD UNIVERSITY

Teachers in the United States are often forced to follow directives that make little sense to them and are far removed from research evidence. One of the initiatives mandated by many school districts that I place high in the category of uninformed policy is the use of timed tests to assess math facts and fluency. Teachers and administrators use these

**IN MY
OPINION** ★

tests with the very best of intentions, but they use them without knowledge of the important evidence that is emerging from neuroscience.



early onset of math anxiety. Indeed, researchers now know that students experience stress on timed tests that they do not experience even when working on the same math questions in untimed conditions (Engle 2002).

In a recent study of 150 first and second graders, researchers measured students' levels of math anxiety, finding that children as young as first grade experienced it and that levels of math anxiety did not correlate with grade level, reading level, or parental income (Ramirez et al. 2013). Other researchers analyzed

[< Back](#)

Anyone having massive anxiety?



Anonymous User

Last Edit: November 17, 2019 10:34 PM

12.2K VIEWS

196



I don't know why, but for the last several months I was having massive anxiety in preparation of upcoming interviews. I'm a veteran engineers (almost 20 years in the industry), and had lost my job this year that was out of my control, and the idea of being asked coding questions drove me crazy. I wish your years of experience, work references, and the fact that you have a college degree actually meant something.

Although I've learned to accept it. I even had to learn relaxation techniques to remain calm. I'll even admit this anxiety has caused me some physical health issue.

Its strange because I've always been a highly productive and outstanding engineering in any role that I've taken (large company or startup).

In any case, I'm curious to hear if you've had this problem. How did you deal with it? How do you see these interviews?

anxiety

stress

Comments: 53

Best

Most Votes

Newest to Oldest

Oldest to Newest

<https://leetcode.com/discuss/career/343147/anyone-having-massive-anxiety>

Post



cool_shark

★ 297

Last Edit: July 25, 2019 5:38 AM

I also consider myself as a veteran engineer (20 years in the industry). I get calls, emails and LinkedIn requests from big companies and I usually ignore those, and while I'm perfectly happy at my job, this year, I responded to the interview requests, and reluctantly accepted to do an interview for several big companies. I was lucky to pass phone interviews for couple of big companies, and then I had to go for on-site interviews (some required flying). Then, I notice I have an anxiety starting. Practicing LeetCode and studying did not help reducing the anxiety. I've done the preparation (more of a hobby) for almost 6 months including 4 months of LeetCode. In my case, the anxiety was due to my inner telling that I was not ready. I was in a position where, even if I pass an on-site interview, I would not be able to accept the offer however good the offer is. So, I told recruiters to contact me later, and cancel on-site interviews. The anxiety was gone, completely. I decided to do more preparation, and then have interviews when I know I'm ready.

The job market has changed drastically -- In the past, companies were measuring knowledge of design patterns (Gang of Four), general software concepts and basic coding skills. These days, even an unknown company tries to ask questions from LeetCode, Cracking the Coding Interview or EPI. and system design (Design Twitter in 20 minutes - Duh!). In other words, the bar has been raised very high, and the interview became like a college entrance exam. It requires dedicated time to prepare for interview. The question is how much preparation is needed. Some people brag that they only solved 80 questions in LeetCode and got a job at Facebook, etc., and it might be possible if the person had PhD specialized in algorithms. But for the rest of us who have daily jobs and became rusty with algorithms and interview crap, I honestly think it requires *minimum six months to two years* of grinding to pass interview at big tech companies (your mileage may vary). You may take a short cut. but what that means is that you're underprepared. I think your massive

How do we tackle the tension between problem-solving (which needs time) and a severe time limit?

Approach 1

Mug up solutions to ~ 2000 Leetcode questions and hope every question you will face will be an exercise?

https://www.reddit.com/r/cscareerquestions/comments/7nkk10/feel_like_im_just_trying_to_memorize_the/

Posted by u/cs_throw_away110 1 year ago 📄

↑
14
↓

feel like im just trying to memorize the solutions from leetcode?

hi guys (and gals), I've been doing leetcode for the past two month-ish.

At first, the struggle was real but now I can comfortably solve easy and some mediums (depending on the condition & questions). However, it seems that a lot of medium questions have their own tricks to solve and I just feel like I'm just trying to memorize these tricks rather than figuring out the solution myself. Is this normal? The problem with this is that if I come across a completely new question (that requires a new trick to solve), I am way off the tangent when I'm trying to solve the question on my own and feel like there would be just too many to memorize.

I've already seen other posts on how they went through this process and landed their dream job but they did not really specify on this topic; was wondering if I could get your experience and 2 cents, cheers!

💬 9 Comments ➦ Share 📌 Save 🚫 Hide 🚩 Report

Approach 1

Mug up solutions to ~ 2000 Leetcode questions and hope every question you will face will be an exercise?

Needs too much time and you're still dependent on luck.

Leetcode HackerRank Computer Programmers Algorithms +2 ✎

After practicing algorithms questions on LeetCode and HackerRank for 13 hours a day for 8 months, I still can't pass any interviews. What's wrong with my method, and what should I do?

🔗 Answer 🔔 Follow · 74 ➔ 👤 Request 💬 3 ⬇️ 📱 🐦 ➦ ...

Doesn't help you become a better engineer.

Leetcode Software Engineers Software Engineering Career Advice ✎

**Why is LeetCode so important for a software engineer?
Why don't universities teach us how to solve LeetCode problems?**

Approach 2

Become a problem-solver and develop *inner confidence*, while getting exposed to popular interview questions grouped by coding patterns.

At IK, you should be learning problem-solving (for new problems) and coding fluency (for exercises)

Learn to tackle questions you have not seen before. Be prepared for the worst-case.

We will be giving you popular interview questions as problems. Avoid looking up solutions and train yourself to become a problem-solver when you see a new question.

Many popular interview questions (problems) will become exercises. But do not forget the problem-solving path you followed to first solve it.

Once a problem has become an exercise, try to improve your coding fluency on it (coding patterns help here).

Problem-solving mindset

Mental discipline: Long, wild-goose chases and open-ended experimentation.

- Timescale of concentration required needs to increase.
- Failure: Investigation is *always* worthwhile even if it leads to a dead-end.

Emotional attitude: Mountaineer vs Gym rat

"The explorer is the person who is lost"

Problem-solving is a craft that can be learnt

- Build background (playground) knowledge
- Learn how other people solved problems
- Active engagement



912. Sort an Array

Medium 473 280 Add to List Share

Given an array of integers `nums`, sort the array in ascending order.

Example 1:

Input: `nums = [5,2,3,1]`

Output: `[1,2,3,5]`

Example 2:

Input: `nums = [5,1,1,2,0,0]`

Output: `[0,0,1,1,2,5]`

Constraints:

- `1 <= nums.length <= 50000`
- `-50000 <= nums[i] <= 50000`

$n \rightarrow \infty$

Is this a problem or an exercise?

How fast can you code this up?



Selection / Bubble Sort : $O(n^2)$

Insertion Sort : $O(n^2)$

- small arrays
- almost sorted

Merge Sort — $O(n \log n)$ worst case, stable, uses aux space

Quick Sort — $O(n \log n)$ average case,
not stable, in-place
↳ empirically fast

Heap sort — $O(n \log n)$ worst case, in-place,
not stable

Counting Sort — narrow range, very few unique elements

Radix Sort — integer arrays (lots of duplicates)

$O(n)$

Space complexity

- Input space
- Output space
- Aux space
- Implicit stack space

Selection Sort (Time limit exceeded)

```
8      #Goal: Sort an array called "nums" which has n integers: nums[0...n-1]
9      #Selection Sort
10
11     #for i in 0 to n-1
12     for i in range(len(nums)):
13         #ith iteration: Find the minimum element from index i to n-1, and put it in index i
14         minelementindex = i
15         minelement = nums[i]
16         for m in range(i+1, len(nums)):
17             if nums[m] < minelement:
18                 minelement = nums[m]
19                 minelementindex = m
20
21         #At this point, the index of the minimum element in nums[i..n-1] is known to be minelementindex
22         #Put it in index i, by swapping it with whatever is there
23         nums[i],nums[minelementindex] = nums[minelementindex],nums[i]
24
25     return nums
```

Bubble Sort (Time limit exceeded)

```
8      #Goal: Sort an array called "nums" which has n integers: nums[0...n-1]
9      #Bubble sort
10
11     #for i in 0 to n-1
12     for i in range(len(nums)):
13         #ith iteration: Find the minimum element from index i to n-1, and put it in index i
14         #Find the min element by scanning the array from index n-1 to index i+1, and swapping the current element
with the element to its left, if they are out of order, i.e, bubble the minimum element from right to left
15         for m in range(len(nums)-1, i, -1):
16             if nums[m] < nums[m-1]:
17                 nums[m-1],nums[m] = nums[m],nums[m-1]
18
19
20     return nums
```

Insertion Sort (Time limit exceeded)

```
8      #Goal: Sort an array called "nums" which has n integers: nums[0...n-1]
9      #Insertion Sort
10
11      #for i in 0 to n-1
12  ▾    for i in range(len(nums)):
13          #ith iteration: Keep nums[0...i] sorted. nums[i] may not be in its right place, so insert it wherever it
should go
14          j = i-1
15          curr = nums[i]
16  ▾    while j >= 0 and nums[j] > curr:
17          nums[j+1] = nums[j]
18          j -= 1
19
20          nums[j+1] = curr
21
22      return nums
```

```

9      #Merge sort an array called "nums" which has n integers: nums[0...n-1]
10
11  def helper(A, start, end):
12      #Base case: Subproblem size = 0 or 1 means nothing to do
13      if start >= end:
14          return
15
16      #Recursive case: Subproblem size at least 2, so cut the subarray into two halves and delegate the sorting
work to two other people hired under you
17      #To make the two halves, find the midpoint
18      mid = (start + end)/2
19      #left half = nums[start...mid] and right half is nums[mid+1...end]
20      #Sort left half
21      helper(A, start, mid)
22      #Sort right half
23      helper(A, mid+1, end)
24      #Merge the two sorted subarrays via a two-pointer pass
25      i = start
26      j = mid+1
27      aux = []
28      while i <= mid and j <= end:
29          if A[i] <= A[j]: # "=" in "<=" important for stability
30              aux.append(A[i])
31              i += 1
32          else: #A[i] > A[j]
33              aux.append(A[j])
34              j += 1
35      #Gather up remaining elements from the subarray whose pointer did not jump off the right edge
36      while i <= mid:
37          aux.append(A[i])
38          i += 1
39      while j <= end:
40          aux.append(A[j])
41          j += 1
42
43      #Copy the elements from the temporary aux space back into the original subarray
44      A[start:end+1] = aux
45
46      helper(nums, 0, len(nums)-1)
47      return nums

```

Mergesort: Target < 10 min

Two styles for recursive code

```
function driver(array nums):  
    helper(nums, 0, nums.length-1)  
    return nums  
end driver function
```

```
function helper(array A, int startindex, int endindex):  
    ...code to sort the subarray A[start...end] here...  
end helper function
```

```
function driver(array nums):  
    function helper(array A, int start, int end):  
        ...code to sort the subarray A[start...end] here...  
    end helper function
```

```
        helper(nums, 0, nums.length-1)  
        return nums  
  
end driver function
```

```

9      #Quicksort an array called "nums" which has n integers: nums[0...n-1]
10
11  def helper(A, start, end):
12      #Base case: Subproblem size = 0 or 1 means nothing to do
13      if start >= end:
14          return
15
16      #Recursive case: Subproblem size at least 2, so cut the subarray into two halves and delegate the sorting
work to two other people hired under you
17      #To make the two halves, partition the array around a random value
18      #Pick a random pivot
19      pindex = random.randint(start, end)
20      #move it to the extreme left
21      A[pindex], A[start] = A[start], A[pindex]
22      orange = start
23      for green in range(start+1, end+1):
24          if A[green] < A[start]:
25              orange += 1
26              A[green], A[orange] = A[orange], A[green]
27      #Now move the pivot at the boundary of the orange and green regions
28      A[start], A[orange] = A[orange], A[start]
29      #Pivot is now at the orange index. Split the array into two partitions, and delegate their sorting work to
two people under you
30      helper(A, start, orange-1) #Sort left partition
31      helper(A, orange+1, end) #Sort right partition
32
33  helper(nums, 0, len(nums)-1)
34  return nums

```

Quicksort: Target < 10 min

Heapsort: Target < 5 min

```
8      #Heapsort an array called nums which has n integers: nums[0...n-1]
9
10     heapq.heapify(nums) #Build a minheap on the input array
11     result = []
12     while len(nums) > 0: #Keep popping off successive elements from the minheap until it is empty
13         result.append(heapq.heappop(nums)) #Accumulate the popped elements into a result array
14
15     return result
16
```

```
8      #Counting sort
9
10     c = collections.Counter(nums)
11     result = []
12     for i in range(-50000, 50001):
13         if i in c:
14             for _ in range(c[i]):
15                 result.append(i)
16
17     return result
```

```

8      #Radix sort
9      #Since numbers are in the range -50,000 to 50000, can focus on 16 + 1 = 17 least significant bits
10     #16 lower bits + 1 sign bit
11     #Do two phases/passes of counting sort, 8 bits at a time
12     for i in range(2):
13         bitmask = (255 << (8*i))
14         c = collections.defaultdict(list)
15         for index in range(len(nums)-1, -1, -1): #Reverse order because stability is needed
16             c[nums[index] & bitmask].append(nums[index])
17
18         aux = []
19         for key in range(256):
20             while len(c[key << (8*i)]) > 0:
21                 aux.append(c[key << (8*i)].pop()) #Where there are multiple copies, lowest index gets popped
first, thus maintaining stability
22
23         nums = aux
24
25     negative = []
26     positive = []
27     for n in nums:
28         if n < 0:
29             negative.append(n)
30         else:
31             positive.append(n)
32
33     negative.extend(positive)
34     return negative
35

```

```

16 ▾ class Heap:
17
18 ▾     def __init__(self, array):
19         self.array = [-1] + array #The underlying array contains elements from index 1...n
20         self.size = len(array) #Size is the number of elements stored in the heap
21         self.buildheap() #This builds the heap in O(n) time
22
23 ▾     def parent(self, i): #Returns index of parent. If parent does not exist (i.e, i is 1 or the root, then it returns a 0)
24         return i/2
25
26 ▾     def leftchild(self, i): #Returns -1 if the left child does not exist. Otherwise returns index of left child
27 ▾         if 2*i < len(self.array):
28             return 2*i
29 ▾         else:
30             return -1
31
32 ▾     def rightchild(self, i): #Returns -1 if the right child does not exist. Otherwise returns index of right child
33 ▾         if 2*i + 1 < len(self.array):
34             return 2*i + 1
35 ▾         else:
36             return -1
37
38 ▾     def getsmallerchild(self, i): #Returns the index of the smaller child. If there is no child, then returns -1
39         res = -1
40 ▾         if self.leftchild(i) != -1:
41             res = self.leftchild(i)
42 ▾         if self.rightchild(i) != -1:
43 ▾             if res == -1:
44                 res = self.rightchild(i)
45 ▾             else:
46 ▾                 if self.array[res] > self.array[self.rightchild(i)]:
47                     res = self.rightchild(i)
48         return res

```

```

50 ▾ def buildheap(self):
51 ▾     for i in range(self.size, 0, -1):
52         self.heapify(i)
53
54 ▾ def insert(self, key):
55     self.array.append(key)
56     self.size += 1
57     currindex = len(self.array)
58     parentindex = self.parent(currindex)
59 ▾     while parentindex != 0 and self.array[parentindex] > self.array[currindex]:
60         #While curr node has value smaller than its parent, swap curr node with parent
61         self.array[currindex], self.array[parentindex] = self.array[parentindex], self.array[currindex]
62         parentindex = self.parent(currindex)
63
64 ▾ def heapify(self, index): #Fix the subtree rooted at this index, if heap property violated at the root
65     #If heap property not satisfied at curr node, find smaller of the two children to swap with
66     smallerchildindex = self.getsmallerchild(index)
67 ▾     while smallerchildindex != -1 and self.array[index] > self.array[smallerchildindex]:
68         #While heap property is not satisfied, swap the curr node value with the value of its smaller child
69         self.array[index], self.array[smallerchildindex] = self.array[smallerchildindex], self.array[index]
70         index = smallerchildindex
71         smallerchildindex = self.getsmallerchild(index)
72
73 ▾ def delete(self): #Pop the root element and return it
74 ▾     if self.size == 0:
75         return null
76     #Root needs to be deleted, so to maintain the heap property, swap it with last element and then delete last element
77     self.array[1], self.array[self.size] = self.array[self.size], self.array[1]
78     popped = self.array[self.size]
79     self.array.pop()
80     self.size -= 1
81 ▾     if self.size >= 1:
82         self.heapify(1)
83     return popped
84

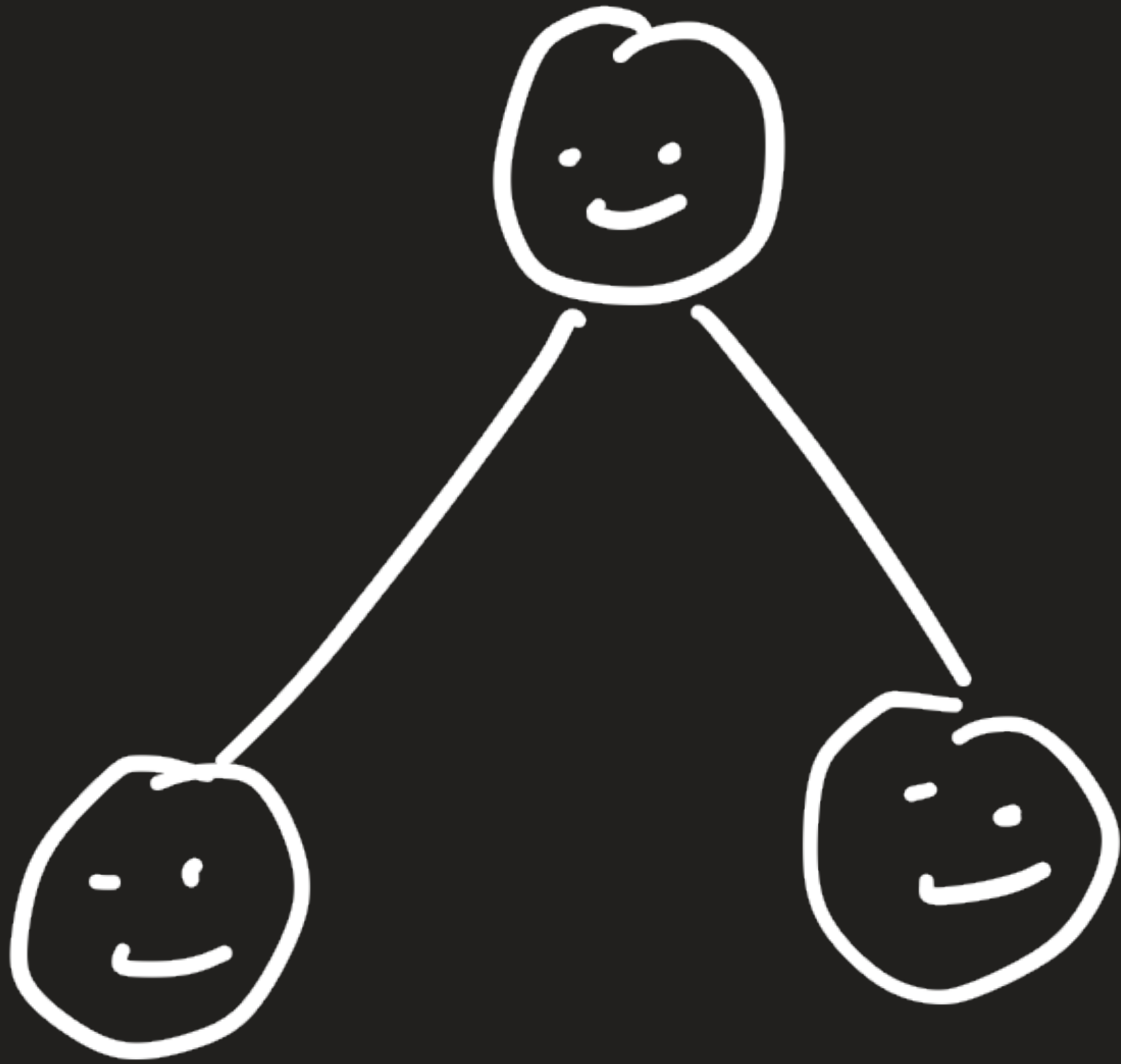
```

```
3 ▼ def sortArray(self, nums):
4     """
5     :type nums: List[int]
6     :rtype: List[int]
7     """
8
9     minheap = Heap(nums)
10
11     result = []
12 ▼ while minheap.size > 0:
13     result.append(minheap.delete())
14 return result
15
```

Takeaways from pre-class videos

1. General algorithm design strategies: brute-force, decrease-and-conquer, divide-and-conquer, transform-and-conquer
2. Algorithm analysis (correctness and efficiency)
 - Big-Oh notation
 - Worst-case, Average-case, Best-case
3. Sorting problem as a case study
 - Merge Sort (Divide-and-conquer), worst-case $O(n \log n)$
 - Quick Sort (Divide-and-conquer), average-case $O(n \log n)$
 - Heap Sort (Transform-and-conquer), worst-case $O(n \log n)$

$$n \rightarrow \infty$$



Part 1



Two sum

You have an array of n numbers and a number *target*. Find out whether the array contains two elements whose sum is *target*.

For example, for the array $[5, 9, 1, 3]$ and $\text{target} = 6$, the answer is yes.

For the same array and $\text{target} = 7$, the answer is no.

BRUTE FORCE

- Exhaustively try all options.

$$nC_2 = \frac{n(n-1)}{2}$$

for i in 0 to n-1:

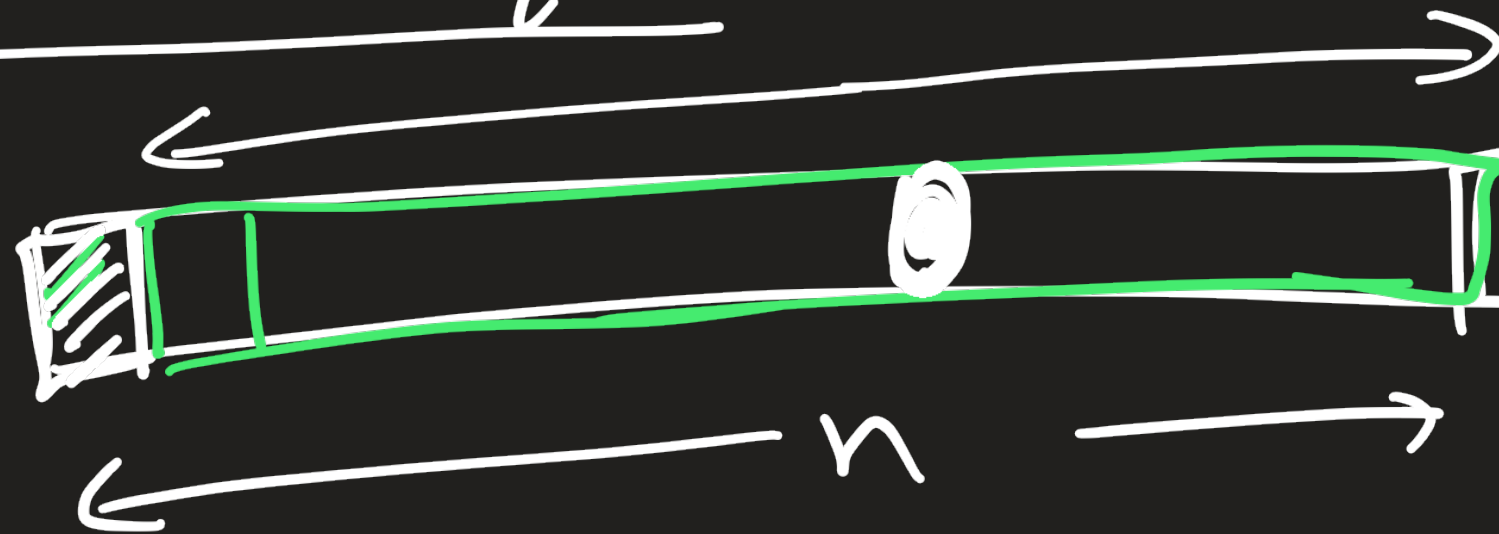
{ for j in i+1 to n-1:

{ if nums[i] + nums[j] == target:
return True

return False

$$\left\{ \begin{aligned} T(n) &= n-1 + n-2 + n-3 + \dots + 1 \\ &= \underline{O(n^2)} \end{aligned} \right.$$

Decrease-and-conquer



Does target - nums[0] exist in the green rectangle?

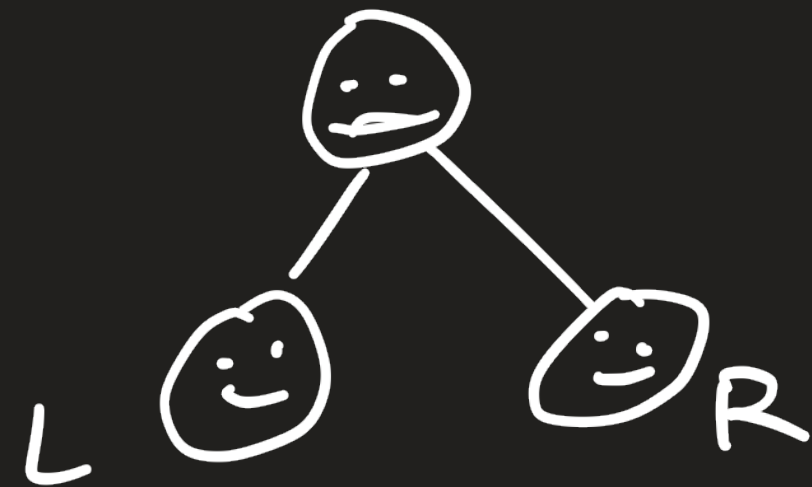
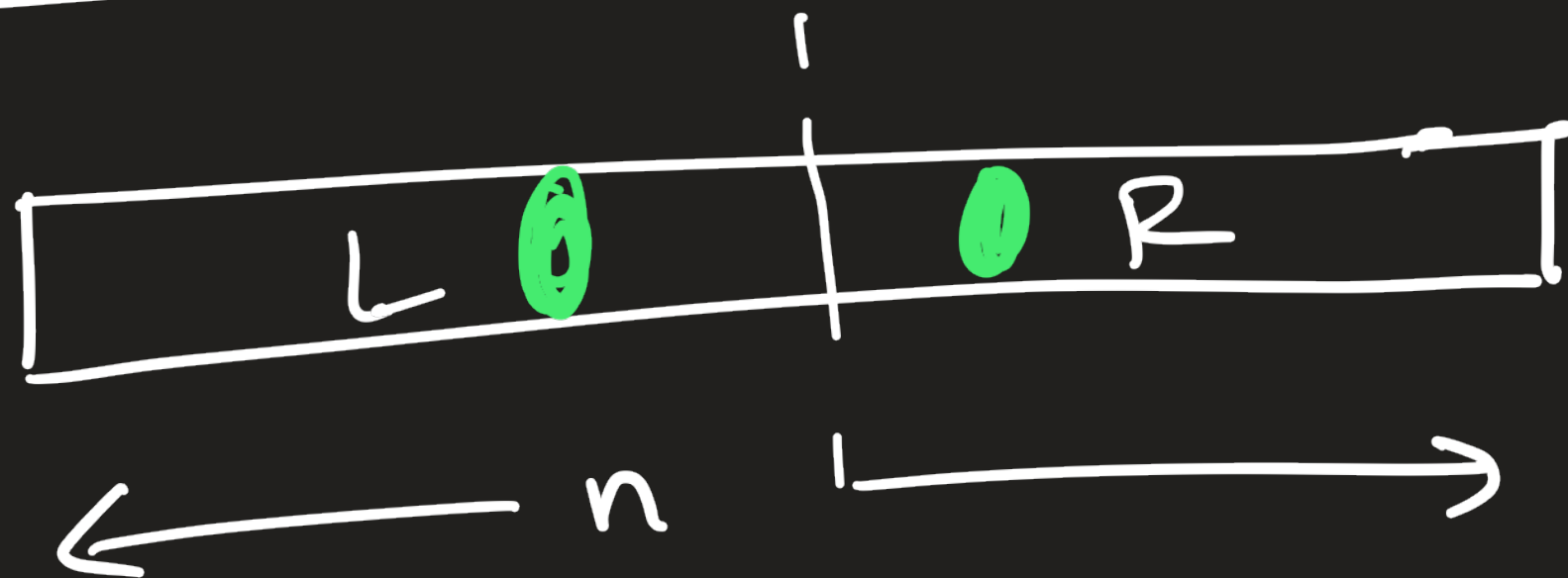
Does target - nums[i] exist in the remaining subproblem?

$$T(n) = n-1 + n-2 + \dots + 1$$

$$T(n) = O(n^2)$$
$$T(n) = \textcircled{cn} + T(n-1)$$



Divide-and-conquer



$$\begin{aligned} T(n) &= T(n/2) + T(n/2) + \underbrace{c(n/2)(n/2)} \\ &= 2T(n/2) + O(n^2) \end{aligned}$$

←

Transform and conquer

Transform the input so that searching for the complementary number can be sped up.

Presorting
(Input rearrangement)
 $O(n \log n)$ to transform

+ $\left\{ \begin{array}{l} n \text{ binary searches} \\ O(n \log n) \end{array} \right.$

$= O(n \log n)$

In-place with quicksort

Can be optimized to $O(n)$

Balanced BSTs
(Representation change)

BST

$\left. \begin{array}{l} \text{Insert} \\ \text{Delete} \\ \text{Search} \end{array} \right\} O(\log n)$

$O(n)$ aux space

$O(n \log n)$ to transform

+ $O(n \log n)$ for n search
 $= O(n \log n)$ ops

TREESET
TREEMAP

HASHSET
HASHMAP

Hash Tables
(Representation Change)

Hash Table $O(n)$ aux space

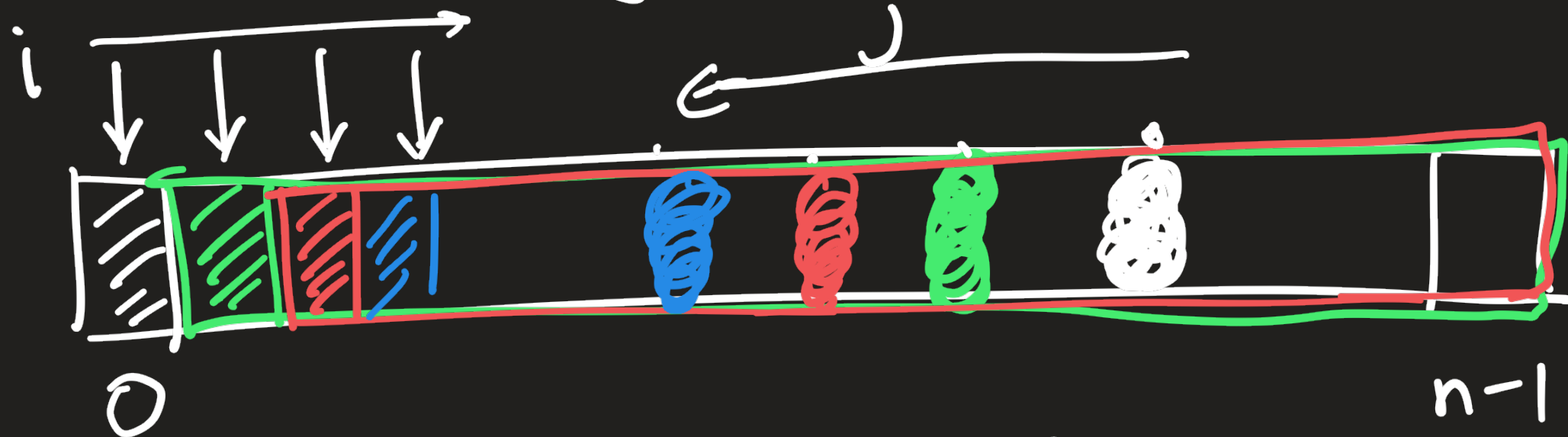
$\left. \begin{array}{l} \text{Insert} \\ \text{Delete} \\ \text{Search} \end{array} \right\} O(1) \text{ expected}$

$O(n)$ time to transform

+ $O(n)$ for n search

ops = $O(n)$

What if the array was already sorted?



TWO POINTERS : $i = 0, j = n-1$

while $i < j$:

{ if $\text{nums}[i] + \text{nums}[j] == \text{target}$:
 return True
 else if $\text{nums}[i] + \text{nums}[j] > \text{target}$:
 $j--$
 else : $// < \text{target}$
 $i++$

→ return False

$T(n) = O(n)$
No aux space.



Space-Time tradeoff between
Presorting & using Hash Tables



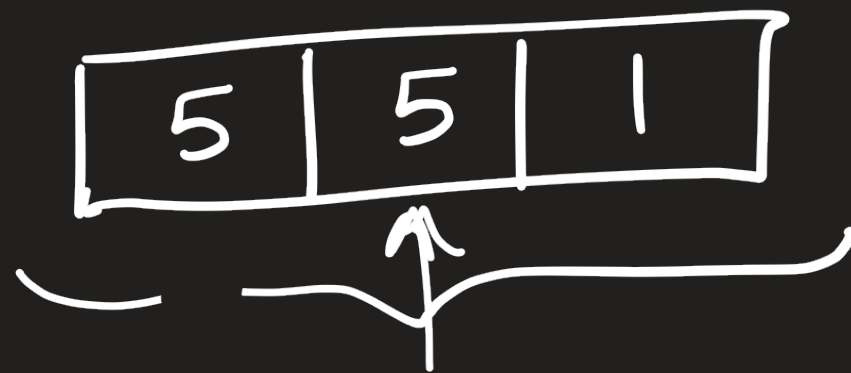
What if duplicates were allowed?

→ Hash Table (if unsorted) :

→ Two-pointer (if sorted)

$\{5, 4, 3\}$
9

Target = 10



$\{5\}$





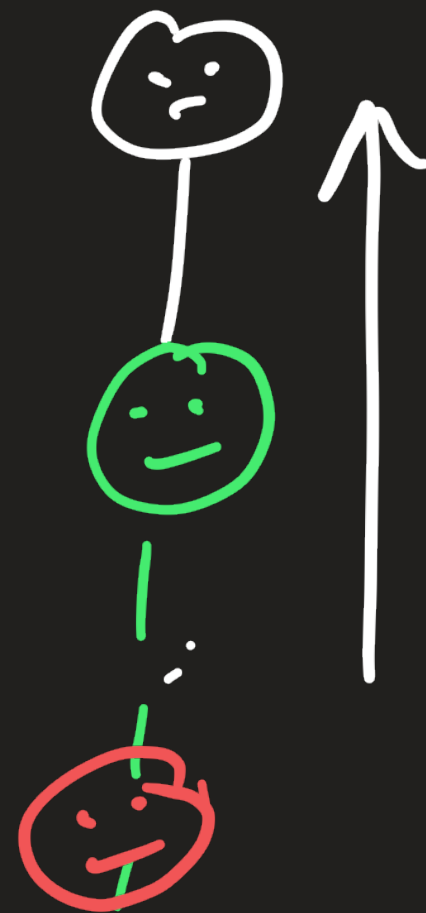
Assumption: All values in my Subordinate's Subproblem are in the hash table.

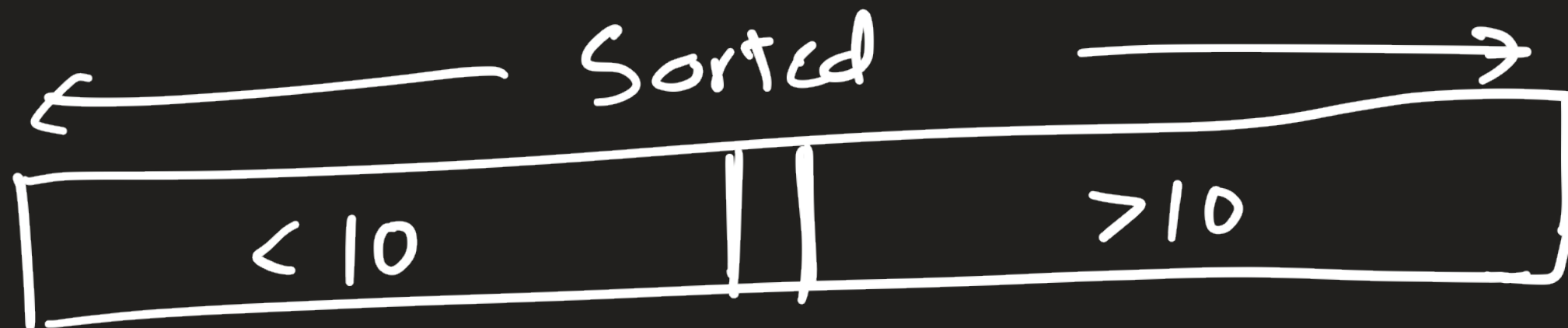
htable $\rightarrow$ An empty dictionary for i in 0 to $n-1$:

{ if target-nums[i] is in htable:
 return True $\rightarrow$ htable[target-nums[i]]
 Insert nums[i] into htable (with value = i) $\rightarrow$ htable[nums[i]] = i

return False

$$T(n) = O(n)$$

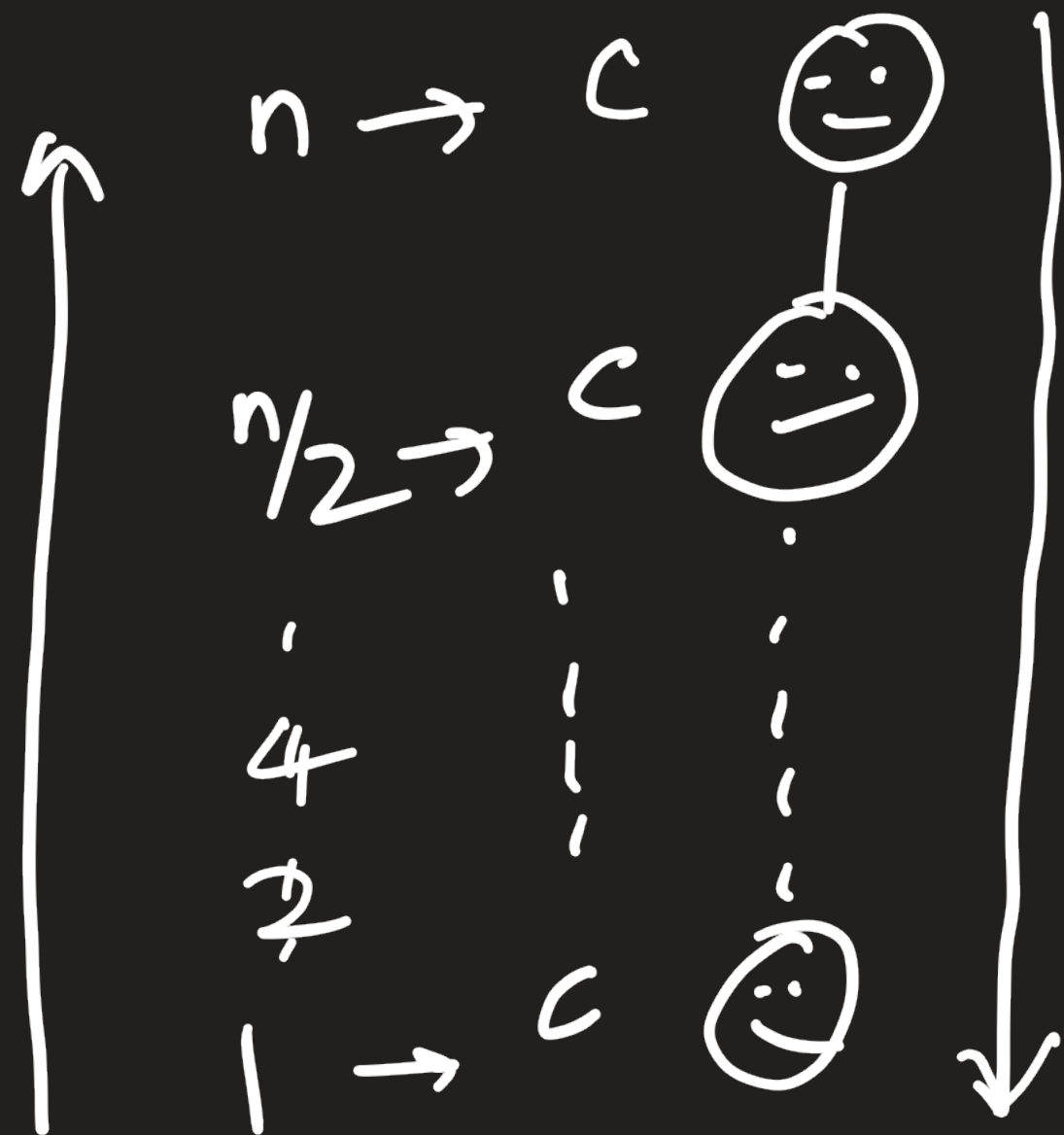




$$\begin{aligned}
 T(n) &= c + T(n/2) \\
 &= c + c + T(n/4) \\
 &= \underline{O(\log n)}
 \end{aligned}$$

$\log_2 n$

BINARY
SEARCH





start = 0, end = length(nums) - 1

while start ≤ end:

mid = (start + end) / 2

if key == nums[mid]:

return "Found"

else if key < nums[mid]:

end = mid - 1

else // key > nums[mid]:

start = mid + 1

return "not found"

Constraints : 0) Array is unsorted $\Rightarrow$ No presorting

1) Return the indices of the two numbers

2) No aux space allowed $\Rightarrow$ No hash tables allowed

Brute force

3 Sum

$T(n) = O(n^3)$
Brute force

- All numbers distinct
- Array unsorted
- Are there 3 numbers adding up to the given target?



$$\begin{aligned} T(n) &= O(n) + T(n-1) \\ &= O(n^2) \end{aligned}$$

1. Two Sum

Easy

👍 11768

💬 406

♡ Favorite

🔗 Share

Given an array of integers, return **indices** of the two numbers such that they add up to a specific target.

You may assume that each input would have **exactly** one solution, and you may not use the *same* element twice.

Example:

```
Given nums = [2, 7, 11, 15], target = 9,
```

```
Because nums[0] + nums[1] = 2 + 7 = 9,  
return [0, 1].
```

167. Two Sum II - Input array is sorted

Easy

👍 1207

💬 487

♡ Favorite

🔗 Share

Given an array of integers that is already ***sorted in ascending order***, find two numbers such that they add up to a specific target number.

The function twoSum should return indices of the two numbers such that they add up to the target, where index1 must be less than index2.

Note:

- Your returned answers (both index1 and index2) are not zero-based.
- You may assume that each input would have *exactly* one solution and you may not use the *same* element twice.

Example:

Input: numbers = [2,7,11,15], target = 9

Output: [1,2]

Explanation: The sum of 2 and 7 is 9. Therefore index1 = 1, index2 = 2.



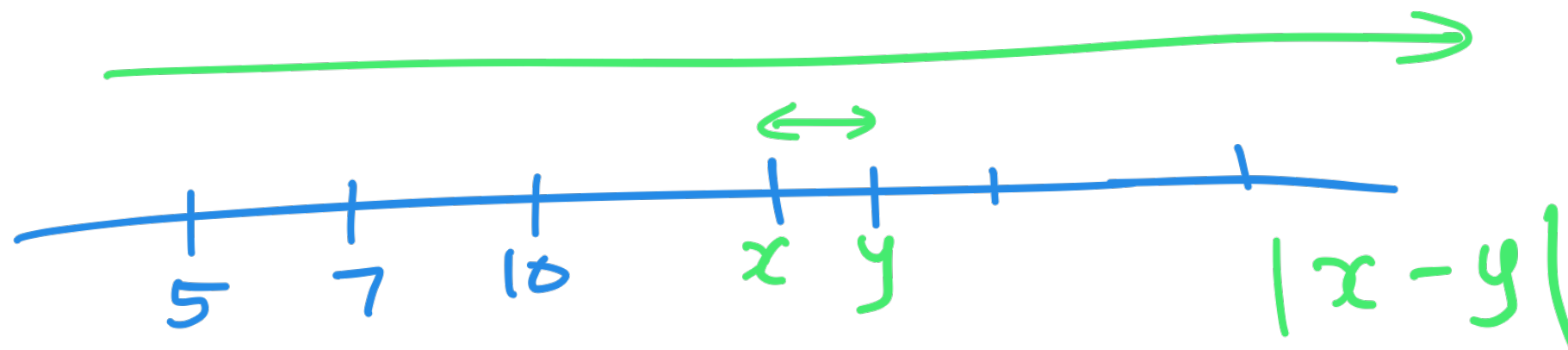
Presorting

Given : An unsorted array of
n numbers

Sorting as a pre-processing step / building block of more complex algorithms

- Repeatedly Presorting + Binary Search $\rightarrow O(\log n) + O(n \log n)$
1. Searching Hash Tables : Search $\rightarrow O(1)$
 2. Closest pair Presorting + Linear scan $\rightarrow O(n \log n)$
 3. Element uniqueness (check for duplicates) : Presorting $O(n \log n) +$
 4. Frequency distribution : Hash Table $O(n)$ Linear scan $O(n)$
 5. Select kth largest/kth smallest/median Hash Table $O(n)$ Linear scan $O(n)$

Presorting $O(n \log n)$ + index lookup



Presorting problems

Common patterns:

Sorting plus binary search

Sorting plus one pass

Sorting plus two-pointer pass

Space-Time
+ Tradeoff with
using hash tables

252. Meeting Rooms

Easy

👍 361

💬 26

♡ Favorite

🔗 Share

Given an array of meeting time intervals consisting of start and end times $[[s_1, e_1], [s_2, e_2], \dots]$ ($s_i < e_i$), determine if a person could attend all meetings.

Example 1:

Input: `[[0,30],[5,10],[15,20]]`

Output: `false`

Example 2:

Input: `[[7,10],[2,4]]`

Output: `true`

15. 3Sum

Medium

9119

956

Add to List

Share

Given an array `nums` of n integers, are there elements a, b, c in `nums` such that $a + b + c = 0$? Find all unique triplets in the array which gives the sum of zero.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Example 2:

Input: `nums = []`

Output: `[]`

Example 3:

Input: `nums = [0]`

Output: `[]`

Constraints:

- $0 \leq \text{nums.length} \leq 3000$
- $-10^5 \leq \text{nums}[i] \leq 10^5$